Setup — Start LocalStack

```
┌──(melon㉿melon)-[~]
└─$ docker run -d --name localstack -p 4566:4566 localstack/localstack
EP='--endpoint-url=http://localhost:4566'
aws $EP logs create-log-group --log-group-name /ccse/app
aws $EP logs create-log-stream --log-group-name /ccse/app --log-stream-name auth

docker: Error response from daemon: Conflict. The container name "/localstack" is already in use by co
ntainer "d973fc62c7c037873170e9fb244f685ba4666ed8350f52ea044c723bab6aefc9". You have to remove (or ren
ame) that container to be able to reuse that name.

Run 'docker run --help' for more information

aws: [ERROR]: An error occurred (ResourceAlreadyExistsException) when calling the CreateLogGroup opera
tion: The specified log group already exists

aws: [ERROR]: An error occurred (ResourceAlreadyExistsException) when calling the CreateLogStream oper
ation: The specified log group already exists
```

Task 1 — Generate Application Logs

```
┌──(melon㉿melon)-[~]
└─$ cat > auth.log << 'EOF'
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5
2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:22 LOGIN_OK user=admin ip=203.0.113.9
2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
EOF
cat auth. log

cat: auth.: No such file or directory
cat: log: No such file or directory

┌──(melon㉿melon)-[~]
└─$ # Your auth.log file is already created (good job!)
# Let's verify it exists:
ls -la auth.log

# View the contents (correct command - no space):
cat auth.log
-rw-rw-r-- 1 melon melon 404 Sep  3 04:48 auth.log
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5
2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9
2025-03-01T09:01:22 LOGIN_OK user=admin ip=203.0.113.9
2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
```

Task 2 — Centralise Logs (Ship to CloudWatch)

```
┌──(melon㉿melon)-[~]
└─$ TS=$(date +%s000)
while IFS= read -r line; do
aws $EP logs put-log-events --log-group-name /ccse/app --log-stream-name auth \
--log-events timestamp=$TS,message="$line" >/dev/null; TS=$((TS+1000)); done < auth.log

# Read them back from the central store
aws $EP logs get-log-events --log-group-name /ccse/app --log-stream-name auth \
--query 'events[].message' --output text

2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5     2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.11
3.9     2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9        2025-03-01T09:01:15 LOGIN_FAIL user=a
dmin ip=203.0.113.9     2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9        2025-03-01T09:01:22 L
OGIN_OK user=admin ip=203.0.113.9       2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
```

Task 3 — Query for Security-Relevant Activity

```
┌──(melon㉿melon)-[~]
└─$ # How many failed logins, and from which IP?
grep LOGIN_FAIL auth.log | awk '{print $4, $5}' | sort | uniq -c

# Distinguish a log (durable record) from an event (a trigger):
# an EVENT would be 'alert: 4 failures from 203.0.113.9' fired in near real time.

      4 ip=203.0.113.9
```

Task 4 — Tamper-Proof (Hash-Chained) Logs

```
┌──(melon㉿melon)-[~]
└─$ # Run the loop and save to auth.chain
PREV=0
while IFS= read -r line; do
    PREV=$(printf '%s%s' "$PREV" "$line" | sha256sum | cut -d' ' -f1)
    printf '%s | %s\n' "$line" "$PREV"
done < auth.log > auth.chain

# NOW view the file (separate command)
cat auth.chain
2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5 | 82da89a49dc1ca7d23b8a59f98d7e557ab36ce0c2d0c6e106fabe76
e1f0acf39
2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.113.9 | 790aef7176d6effe76d077831c071f8500204bf842e7fd8aed
a1b67b2e271a97
2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9 | 1e0b2e8aaf5143fb95070a8e57b009f058f0d37c257d19409b
4131894d29a9a8
2025-03-01T09:01:15 LOGIN_FAIL user=admin ip=203.0.113.9 | 7fb62c66ded511605e22c8db9c4f57c9360aa27309ce65024a
3e5ea35e3b6e94
2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9 | 143253b549a74b9626e910fbe54ca12cb5431a0a4c9c4f2189
ff27a3e2a17e01
2025-03-01T09:01:22 LOGIN_OK user=admin ip=203.0.113.9 | 4cbfab7fecb703cf21f5df81b47dbf3a727c94442b09b714ac4b
faa3584cc638
2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB | ababa787b4bf524d9daddca8c48e4909fc1057
69a6f17574f42cefe8f81233cf
```

```
┌──(melon㉿melon)-[~]
└─$ >....
TAMPER=$(tail -1 auth.tampered.chain | awk -F' \\| ' '{print $2}')

echo "4. Final Hash Comparison:"
echo "   Original: $ORIG"
echo "   Tampered: $TAMPER"
echo ""

# Verify if tampering detected
if [ "$ORIG" ≠ "$TAMPER" ]; then
    echo "❌ TAMPERING DETECTED!"
    echo "   The hash chain proves the log was altered."
    echo ""
    echo "   📊 Evidence:"
    echo "   Original entry:  $(grep EXPORT_DATA auth.log)"
    echo "   Tampered entry:  $(grep EXPORT_DATA auth.tampered)"
    echo ""
    echo "   🔑 The final hash changed, proving tampering!"
else
    echo "✅ No tampering detected"
fi
EOF

# Make it executable
chmod +x check_tamper.sh

# Run it
./check_tamper.sh
4. Final Hash Comparison:
   Original: ababa787b4bf524d9daddca8c48e4909fc105769a6f17574f42cefe8f81233cf
   Tampered: 72f1d53774a3a938fa7bd3a88f67894e5a64055a41ee7511eac53d7bd89d859b

❌ TAMPERING DETECTED!
   The hash chain proves the log was altered.

   📊 Evidence:
   Original entry:  2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
   Tampered entry:  2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=5MB

   🔑 The final hash changed, proving tampering!
```

Task 5 — Detect the Incident (Correlation)

```
┌──(melon㉿melon)-[~]
└─$ IP=203.0.113.9
FAILS=$(grep -c "LOGIN_FAIL.*$IP" auth.log)
SUCCESS=$(grep -c "LOGIN_OK.*$IP" auth.log)
EXPORT=$(grep -c "EXPORT_DATA.*$IP" auth.log)
echo "IP=$IP fails=$FAILS success=$SUCCESS export=$EXPORT"

if [ "$FAILS" -ge 3 ] && [ "$SUCCESS" -ge 1 ] && [ "$EXPORT" -ge 1 ]; then
echo 'ALERT: probable brute-force → compromise → data exfiltration';
fi

IP=203.0.113.9 fails=4 success=1 export=1
ALERT: probable brute-force → compromise → data exfiltration
```

Task 6 — Incident Response